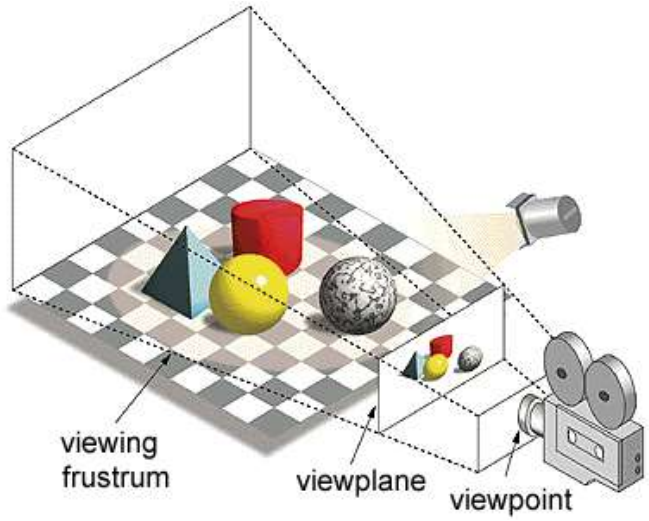
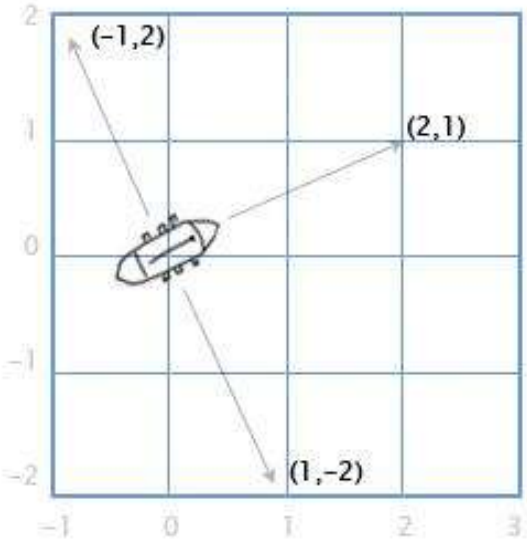
Graphics Pipeline

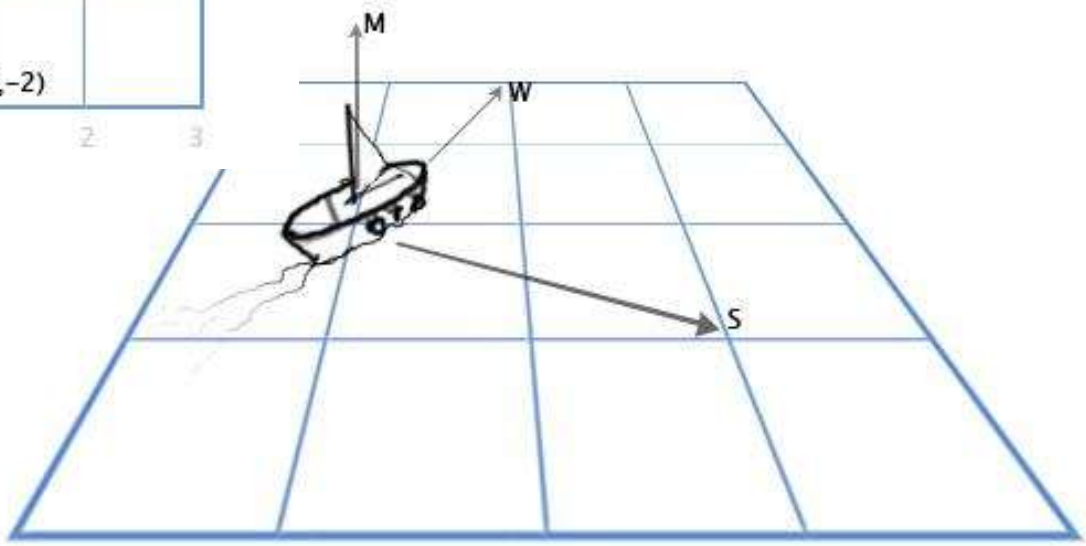
This is a special software/hardware subsystem that efficiently draws 3D primitives in perspective

The basic operations in the pipeline map the 3D vertex locations to 2D screen positions and shade the triangles so that they both look realistic and appear in proper back-to-front order.

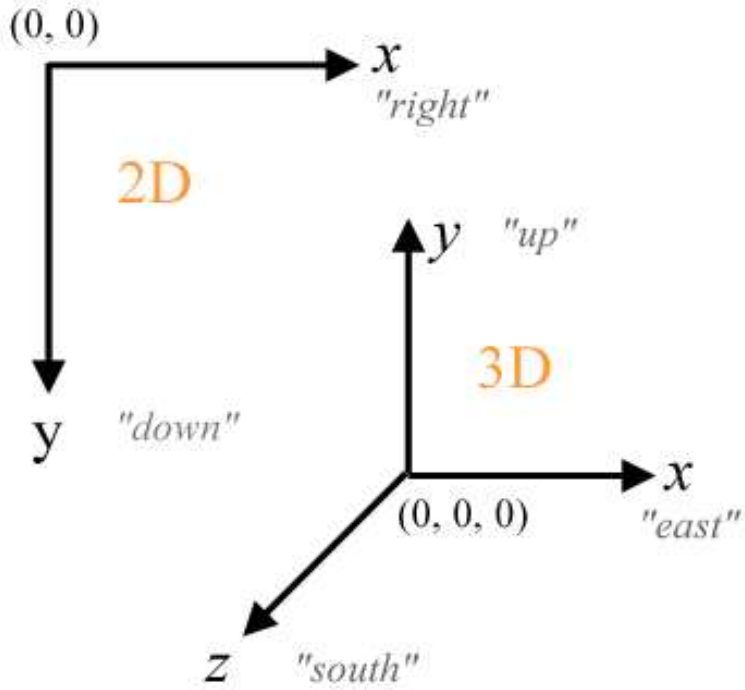


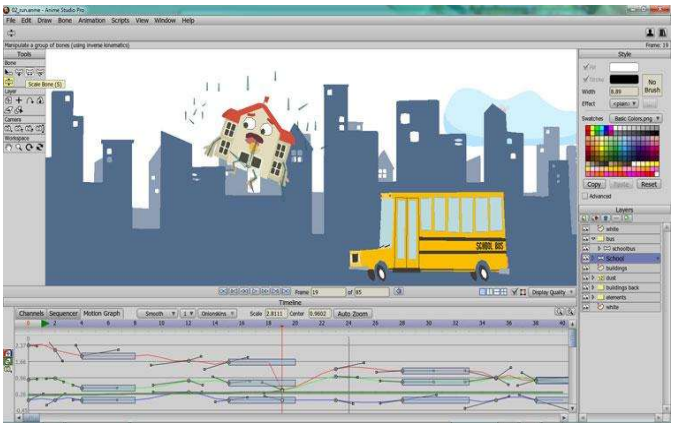
Space

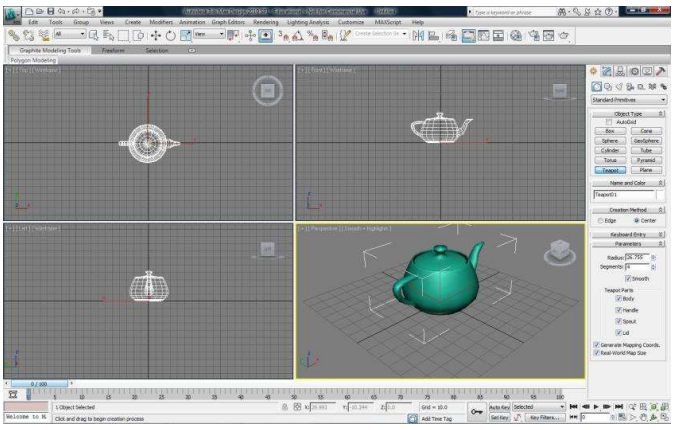




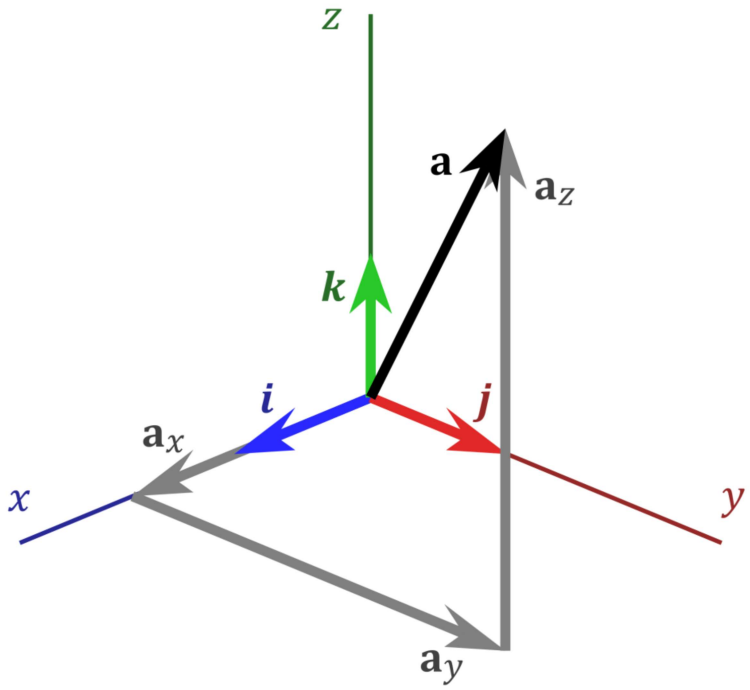
2D and 3D Space







Vector Spaces

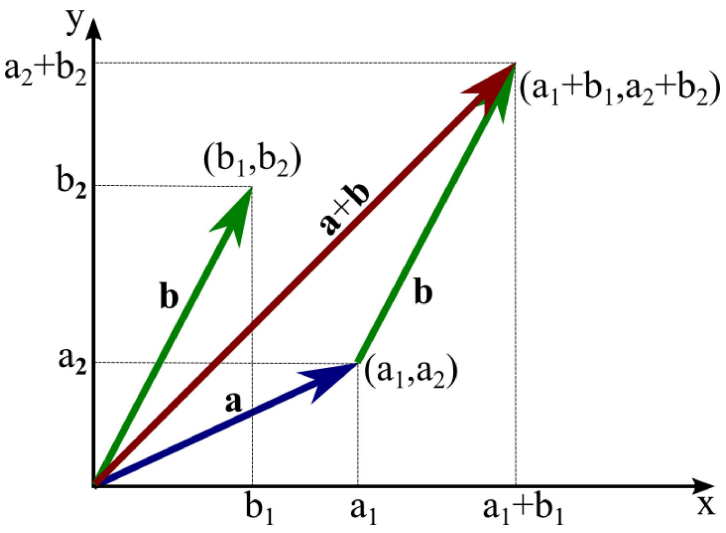


A vector space is a mathematical structure that is defined by a given number of linearly independent vectors, also called base vectors.

Vector Spaces

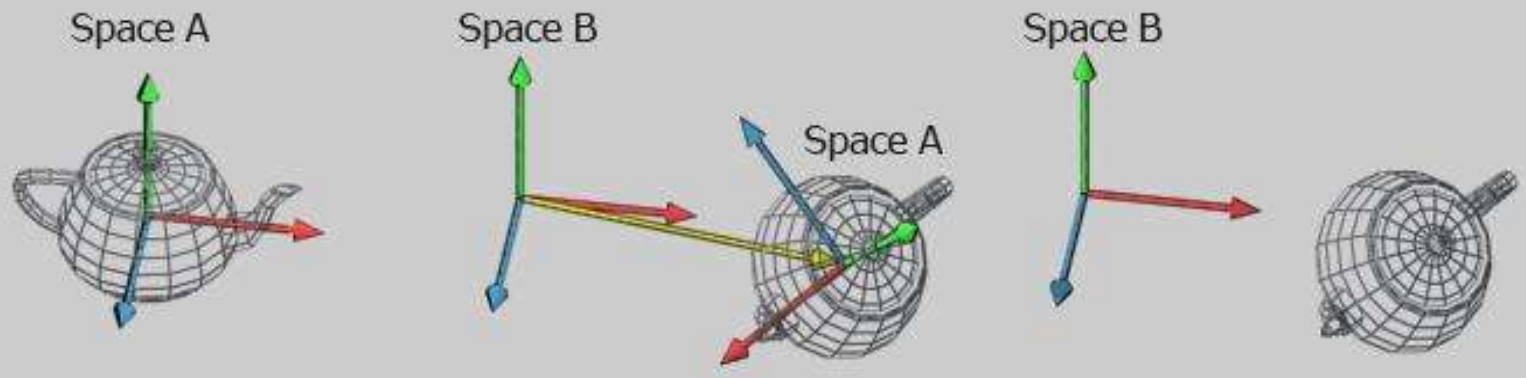
The number of linearly independent vectors defines the size of the vector space, therefore a 3D space has three base vectors, while a 2D space would have two.

These base vectors can be scaled and added together to obtain all the other vectors in the space.

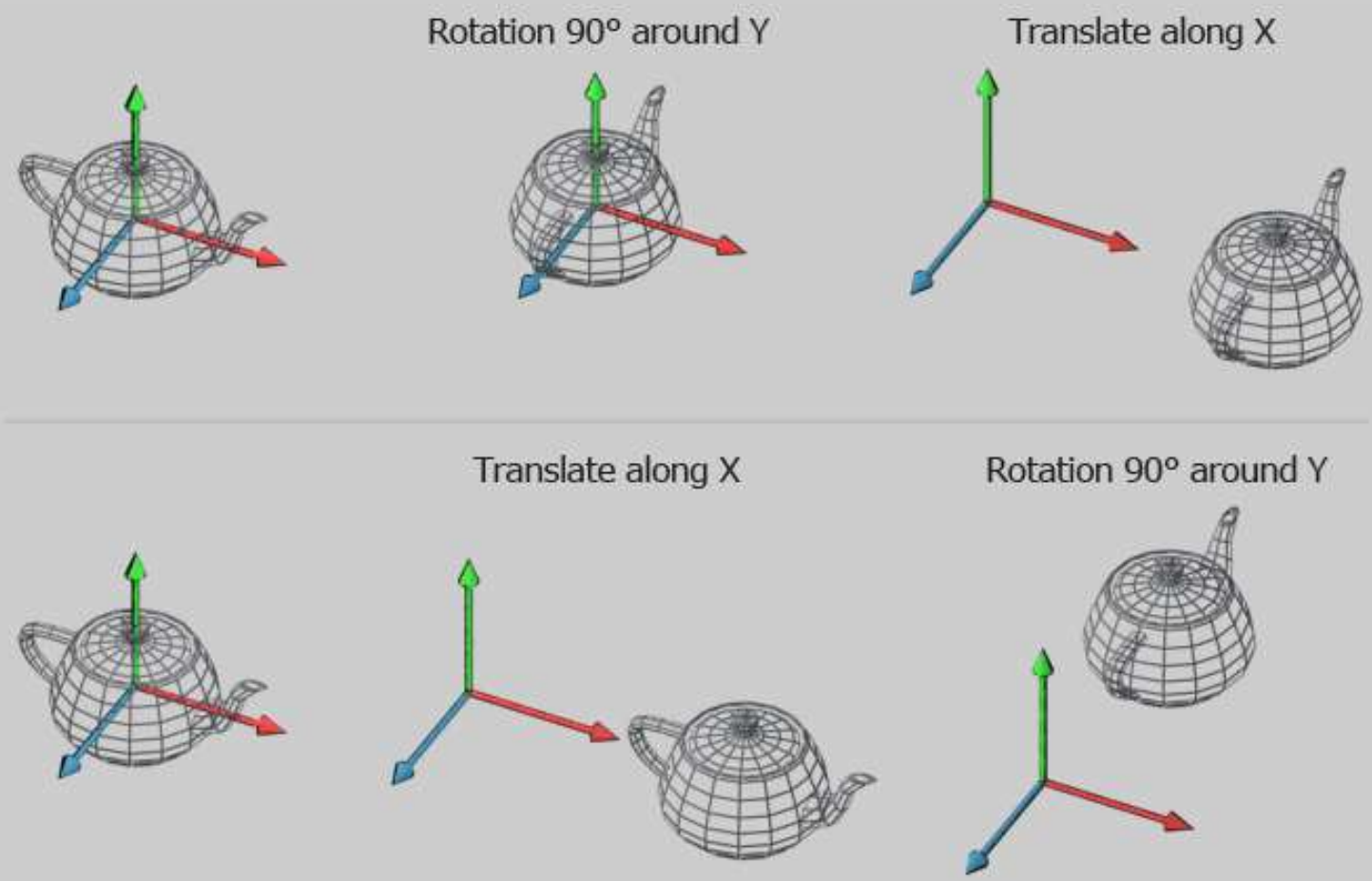


$$\begin{pmatrix} 1 & a & b \\ c & 1 & d \\ e & f & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} x + a \cdot y + b \cdot z \\ c \cdot x + y + d \cdot z \\ e \cdot x + f \cdot y + z \end{pmatrix}$$

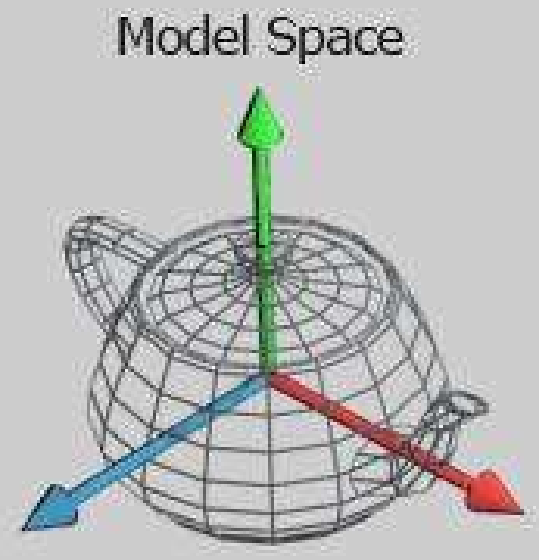
Space Transformations



Same transformations applied in a different order



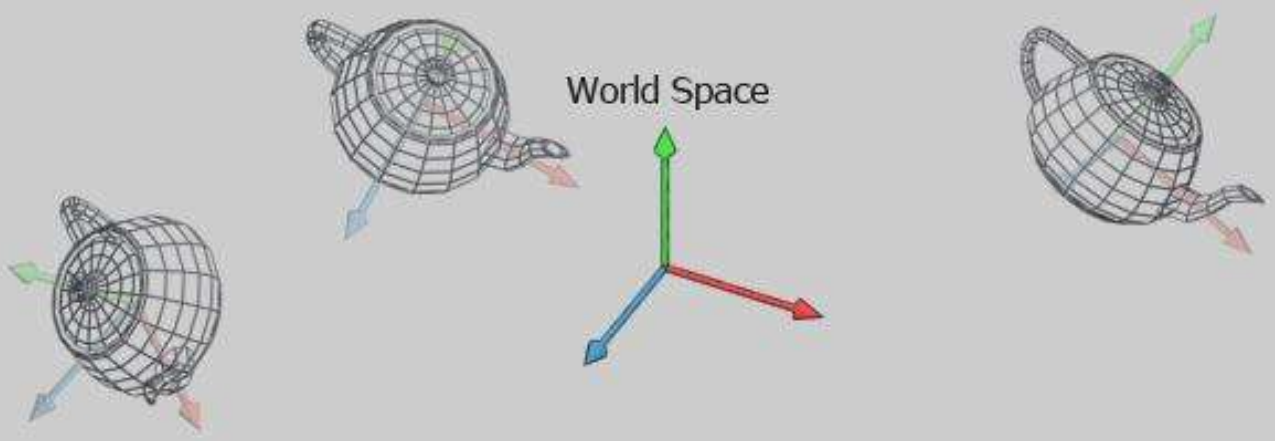
Model Space



Model Space

- What is Model Space?
  - Model Space (also called Object Space) is the local coordinate system of a 3D model.
- It's where the model is originally created—before it’s placed in the world.
- All the vertices and geometry of the model are defined relative to its own origin.
- For example, if you have a teapot model:
  - Its origin (0, 0, 0) might be at the center of the teapot.
  - The spout, lid, and base are positioned relative to that center, not to the camera or the world.

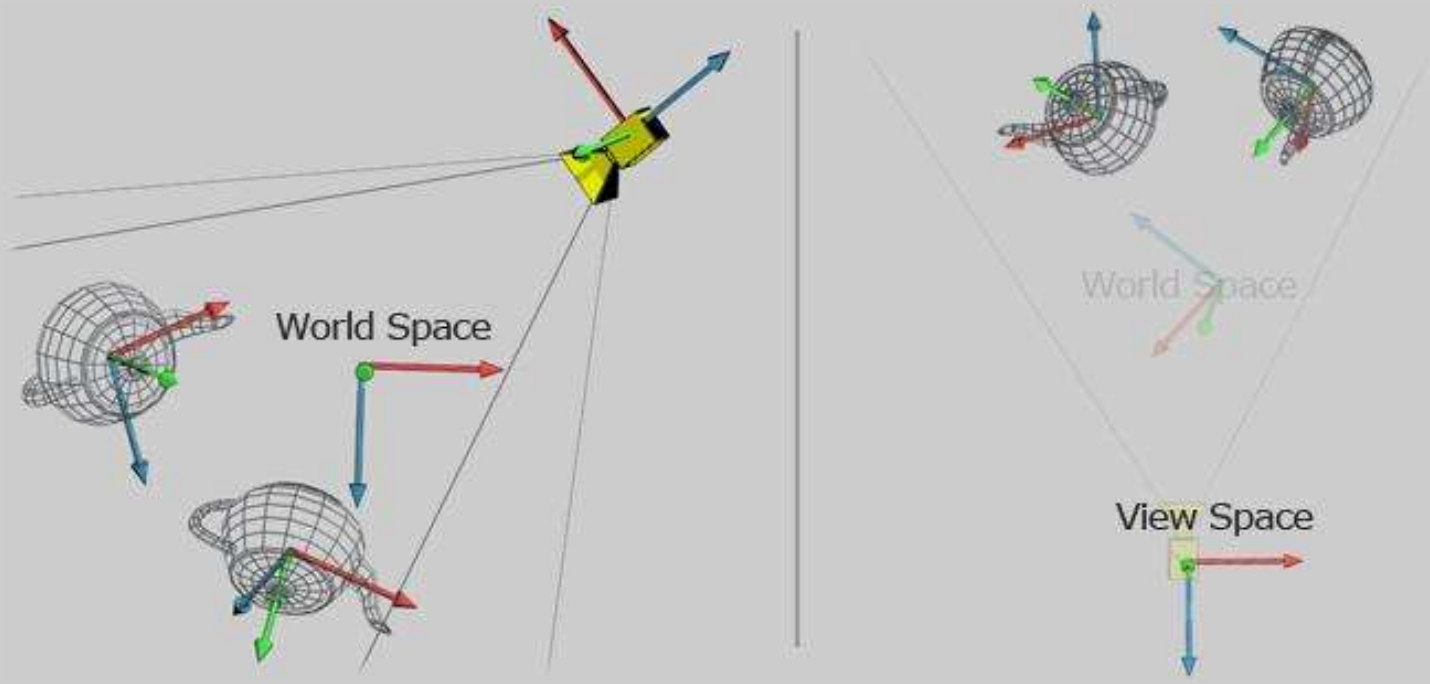
World Space



World Space

- World Space (or World Coordinate System) is a common coordinate system in a 3D scene where all objects are placed.
  - Purpose: It defines the absolute positions and orientations of objects in the scene.
- Each model has its local orientation and position.
- The World Space axes (usually X in red, Y in green, Z in blue) define a universal frame of reference.

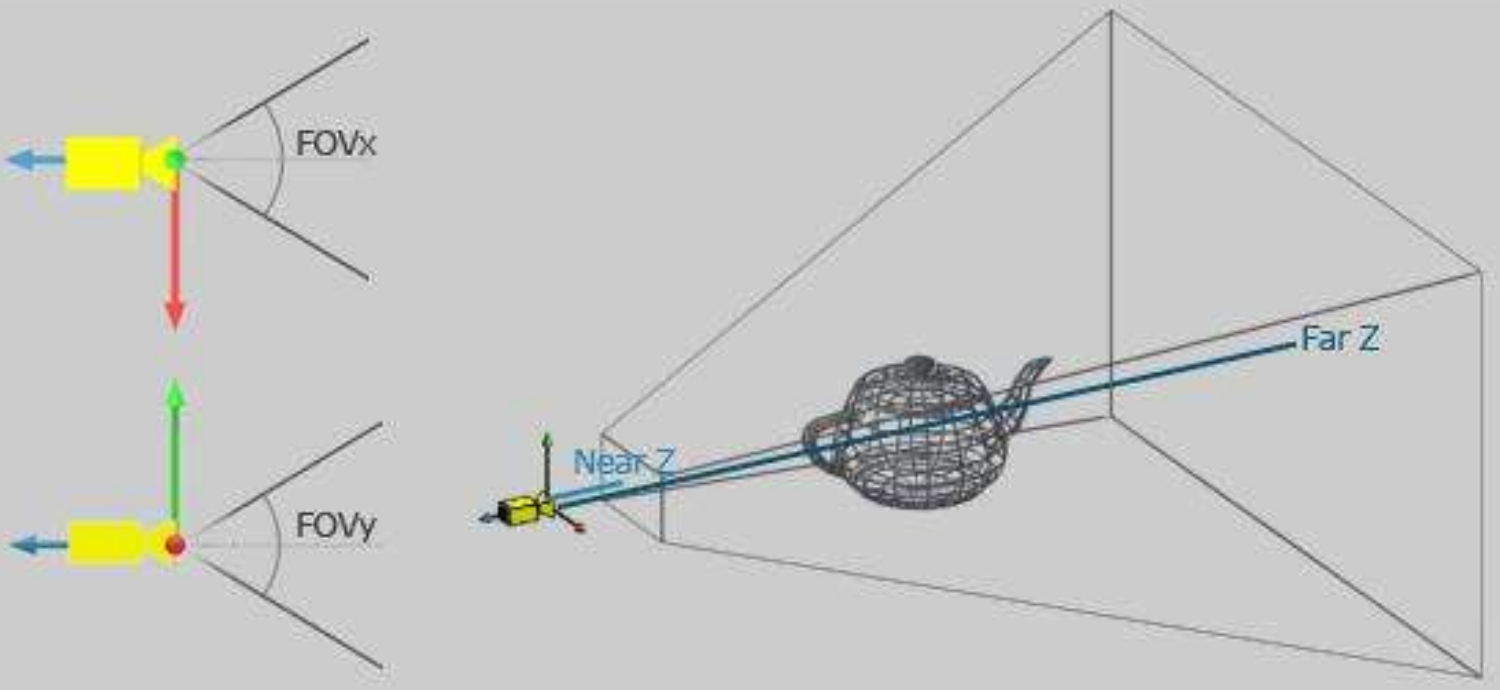
World Space & View Space



View Space

- View Space (Camera Space / Eye Space)
- Definition: View Space is the coordinate system relative to the camera's point of view.
- Purpose: It transforms the world so that the camera is at the origin (0, 0, 0) looking down the -Z axis.
  - All object coordinates are recalculated relative to the camera.

Projection Space



Projection Space

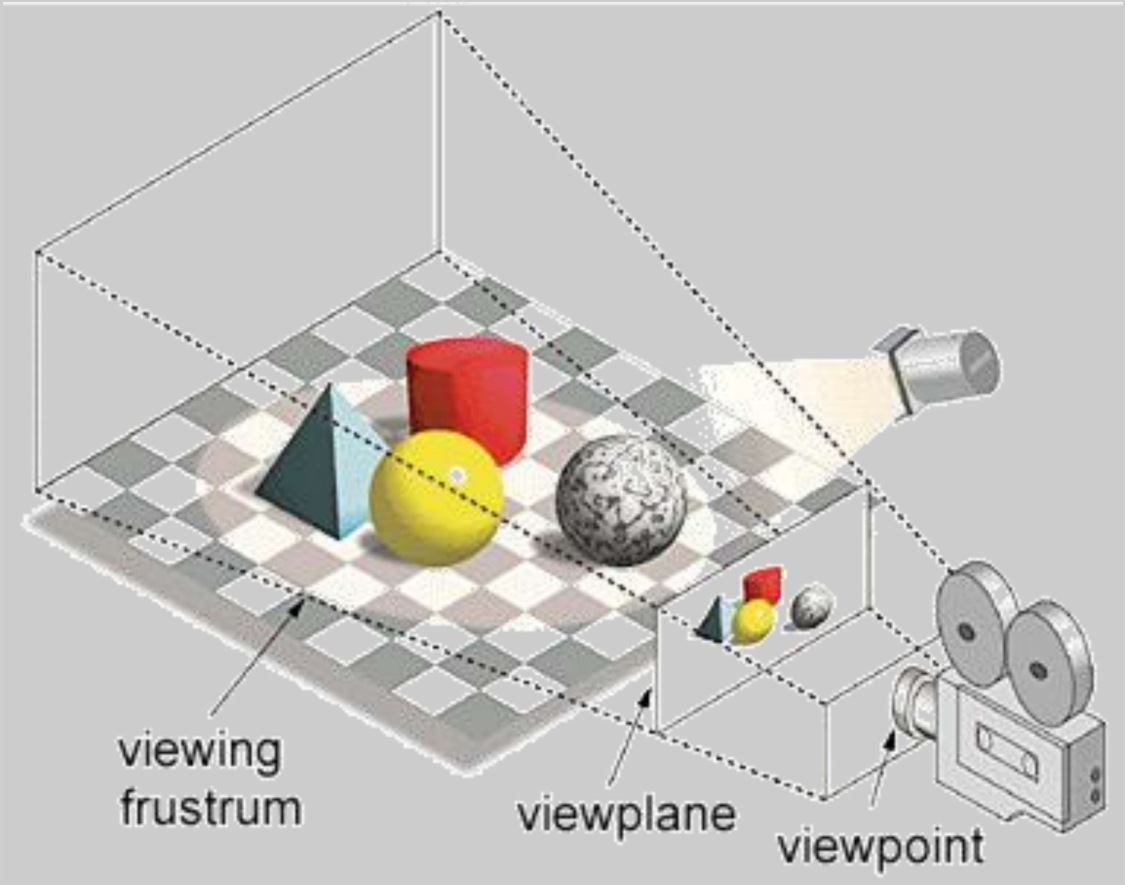
Term	Meaning
FOVx / FOVy	Horizontal and vertical angles of view
Near Z	Closest distance the camera can see
Far Z	Farthest distance the camera can see
Frustum	3D pyramid-shaped volume of visible space
Projection Space	Result of applying a projection matrix

The diagram shows how the perspective projection transforms a 3D scene into Projection Space, or Clip Space

Projection Space

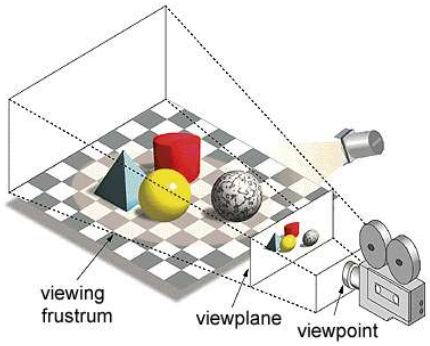
- Projection Space (or Clip Space) is the coordinate system that results after applying the projection matrix to View Space coordinates.
- It is a crucial step in the 3D rendering pipeline because it transforms the 3D scene into a 2D viewable format, ready to be mapped to your screen.

Projection Space



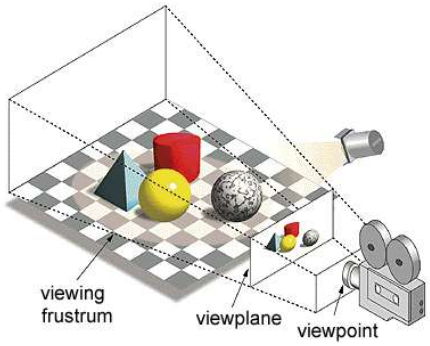
Projection Space - Viewpoint

(Camera Position)



- This is where the **virtual camera** is located — analogous to the eye or lens of a real camera.
- All rays used to render the image **originate from this point**.
- It corresponds to the **origin of the View Space** in computer graphics.

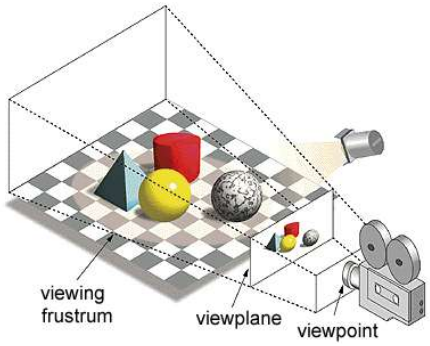
Projection Space - Viewing Frustum



- A **truncated pyramid** shape that represents the **visible space** from the camera's perspective.
- Objects inside this frustum are visible; those outside are **clipped**.
- It is defined by:
  - **Near Plane** , **Far Plane** (farthest visible depth), **FOV (Field of View)**

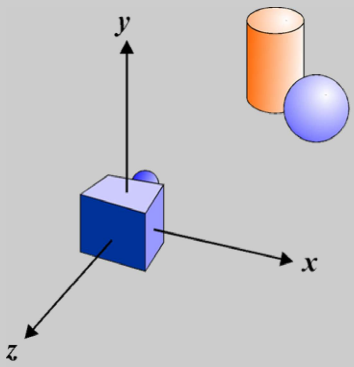
Projection Space – View plane

(Projection Plane)

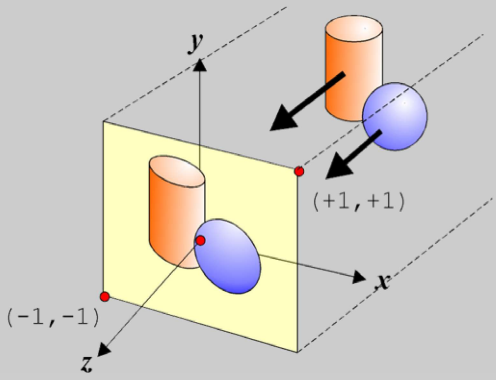


- Think of this as the **virtual screen** inside the 3D world where the image is projected.
- Every object is **projected onto this plane** using the camera's perspective.
- It's conceptually similar to how a real camera focuses light onto a film or sensor.

Viewport Coordinate System

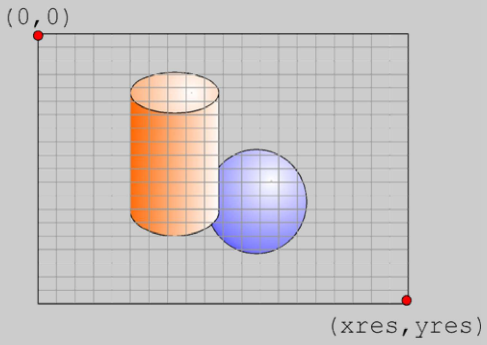


World/Object Coordinates



Normalized Device Coordinates (NDC)

Device/Screen Coordinate System



Screen Coordinates (also called **Device Coordinates**)

Transformation Flow

- To render a 3D scene, coordinates of every object go through a series of transformations:
- Model Space → World Space
  - Converts object-local coordinates to world position.
- World Space → View Space
  - Transforms world coordinates into coordinates relative to the camera.
- View Space → Clip/Projection Space
  - Applies perspective projection (not shown here).
- Clip Space → Screen Space
  - Maps the projected 3D points to 2D screen pixels.

Transformation Flow

Stage	Description	Transformation Used
Model Space	Local coordinates of an object	—
World Space	Object in the 3D world	Model Matrix
View Space	Object relative to the camera	View Matrix
Projection Space	Applies perspective or orthographic view	**Projection Matrix**
Screen Space	2D pixel coordinates	Viewport transformation

3D Graphics Pipeline

- Creating objects/scene – 3D modeling
  - define objects in local coordinate system
  - transform objects to world coordinate system

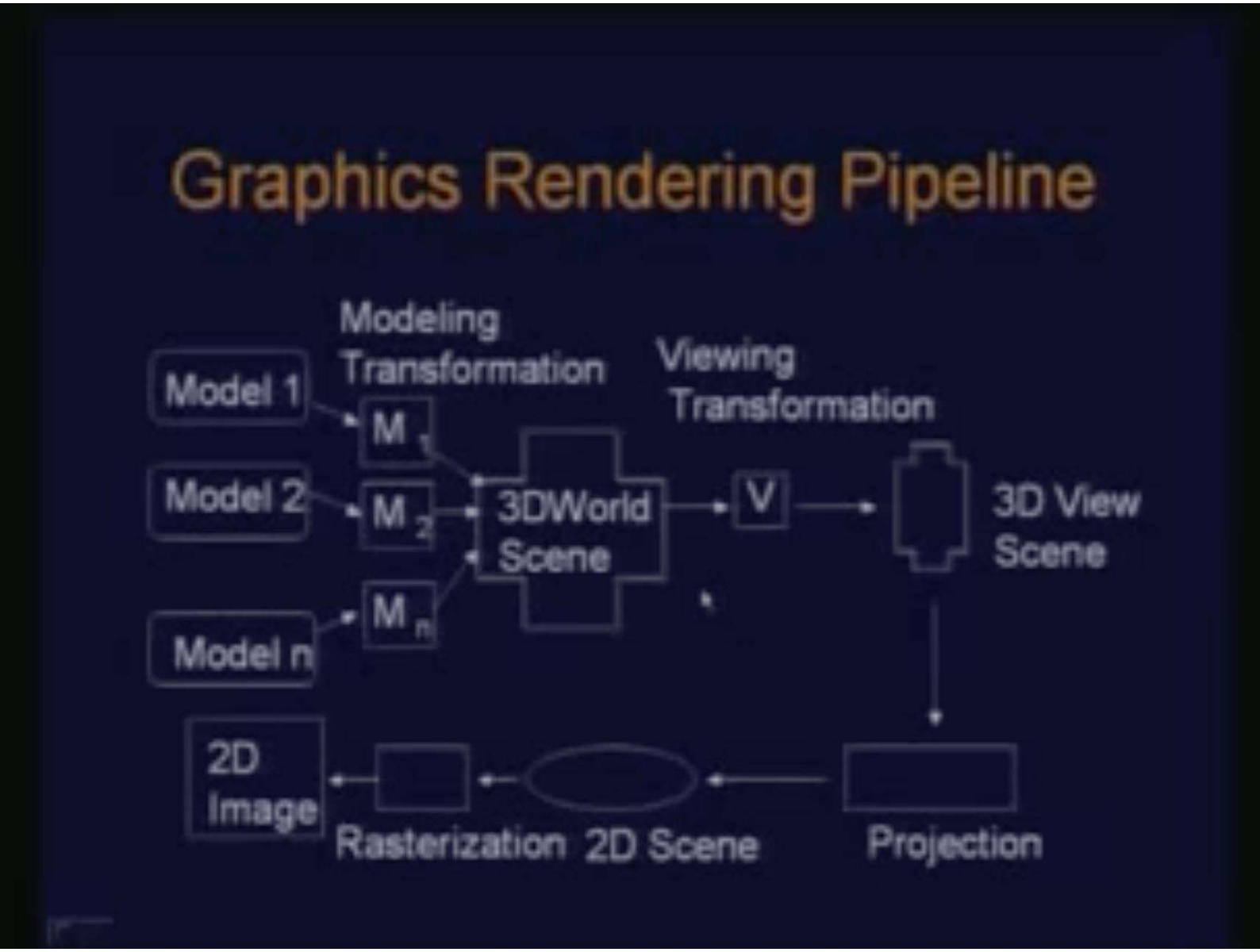
- Determining which part of the objects/scene are visible
  - camera model (position, direction, focal length) to define viewing volume
  - transform objects to camera coordinate system
  - clipping against view volume to determine visible objects
  - projection to obtain 2D representation of objects

- Determining which pixels should be filled
  - Rasterisation – from vectors/polygons to pixels
  - Hidden Surface Removal - determine which objects in the view volume should appear in image at each pixel

- Determining what color/texture/lighting it is
  - lighting models and texture mapping

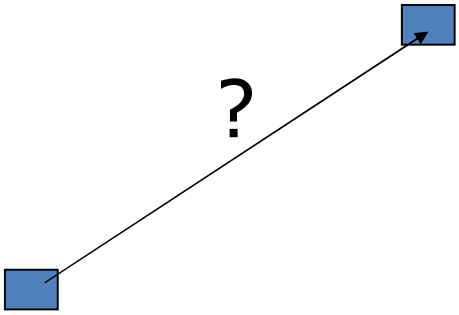
3D Viewing Pipeline

1. The viewing transformation takes vertices in the world coordinates into viewing coordinates.
2. The clipping operation eliminates the graphics outside the specified view volume.
3. The appropriate projection transformation to project 3D graphics in the view volume onto the view plane should be carried out.
4. Finally, transform the view window contents on to the viewport and then transforming these coordinates into physical device coordinates.


Graphics Rendering Pipeline
Modeling Transformation
Viewing Transformation
Model 1
Model 2
Model n
M 1
M 2
M n
3DWorld Scene
V
3D View Scene
2D Image
Rasterization
2D Scene
Projection

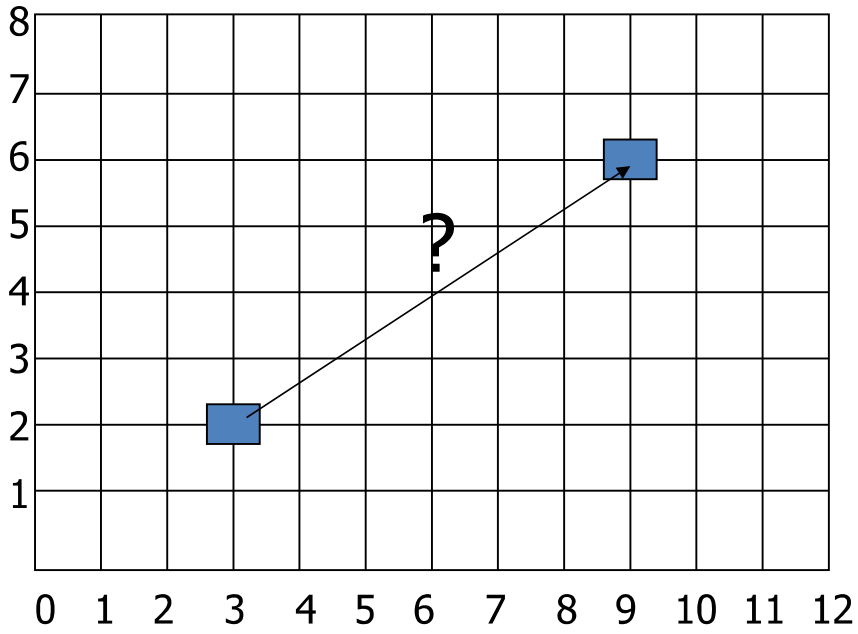
Drawing algorithms

E.g. Line drawing



Drawing algorithms

E.g. Line drawing



Line: (3,2) -> (9,6)

Which intermediate pixels to turn on?

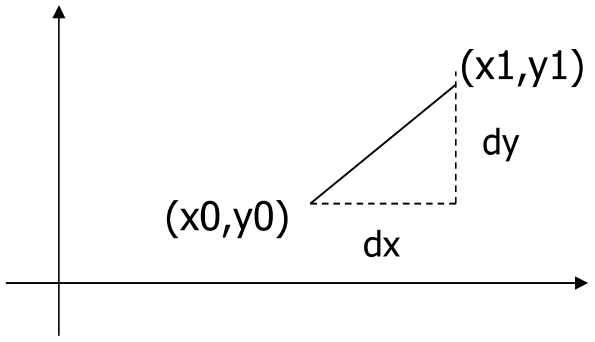
Drawing algorithms

E.g. Line drawing

- Slope-intercept line equation
  - y = mx + b
  - Given two end points (x0,y0), (x1, y1), how to compute m and b?

$$m = \frac{dy}{dx} = \frac{y1 - y0}{x1 - x0}$$

$$b = y0 - m * x0$$



Drawing algorithms

E.g. Line drawing

- Numerical example of finding slope m:
- (Ax, Ay) = (23, 41), (Bx, By) = (125, 96)

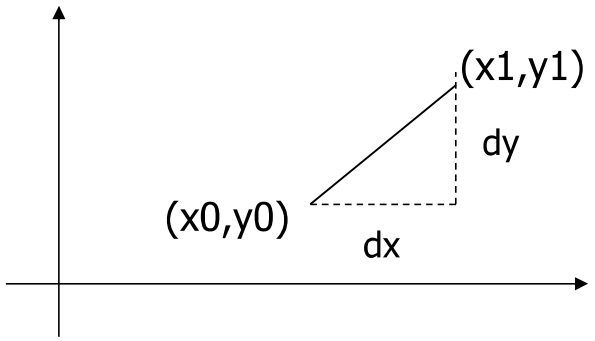
$$m = \frac{By - Ay}{Bx - Ax} = \frac{96 - 41}{125 - 23} = \frac{55}{102} = 0.5392$$

Drawing algorithms

E.g. Line drawing

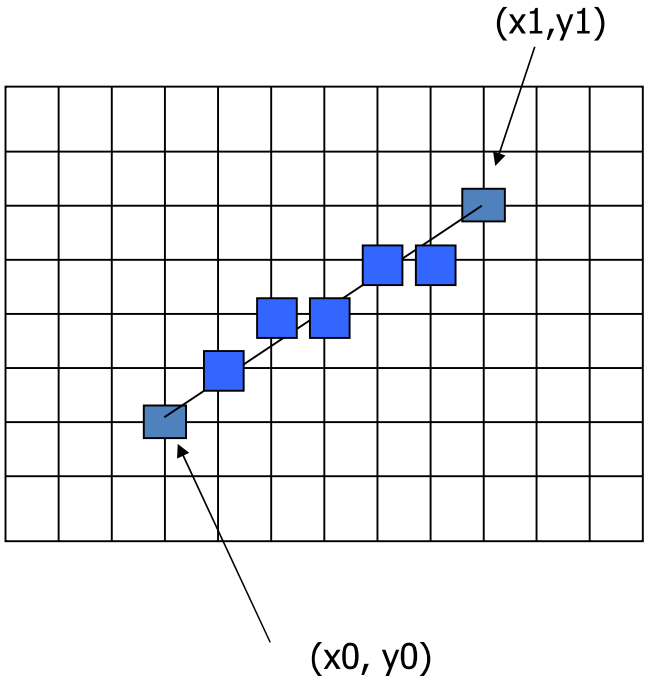
Digital Differential Analyzer (DDA): Line Drawing Algorithm

Walk through the line, starting at (x0,y0)
Constrain x, y increments to values in [0,1] range
Case a: x is incrementing faster (m < 1)
    Step in x=1 increments, compute and round y
Case b: y is incrementing faster (m > 1)
    Step in y=1 increments, compute and round x



DDA Line Drawing Algorithm (Case a: m < 1)

$$y_{k+1} = y_k + m$$



x = x0 y = y0

Illuminate pixel (x, round(y))

x = x0 + 1 y = y0 + 1 * m

Illuminate pixel (x, round(y))

x = x + 1 y = y + 1 * m

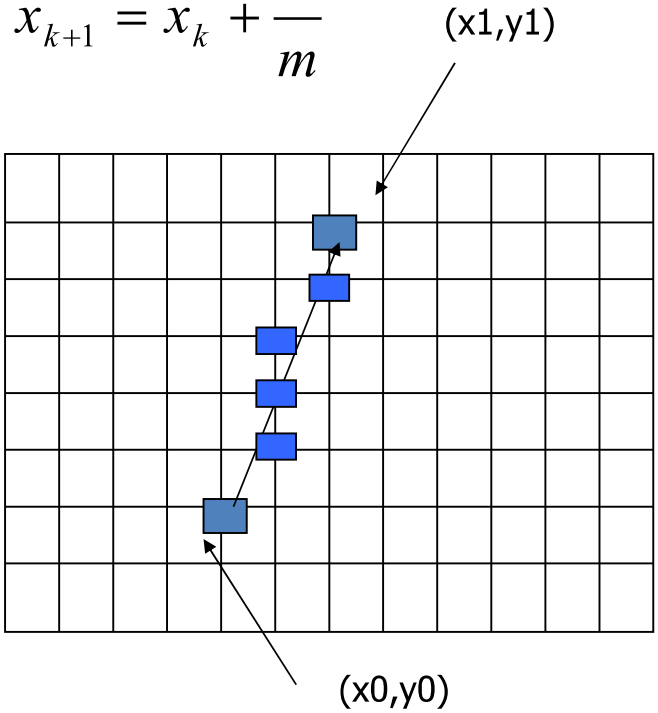
Illuminate pixel (x, round(y))

…

Until x == x1

DDA Line Drawing Algorithm (Case b: m > 1)

$$x_{k+1} = x_k + \frac{1}{m}$$



x = x0 y = y0

Illuminate pixel (round(x), y)

y = y0 + 1 x = x0 + 1 * 1/m

Illuminate pixel (round(x), y)

y = y + 1 x = x + 1 /m

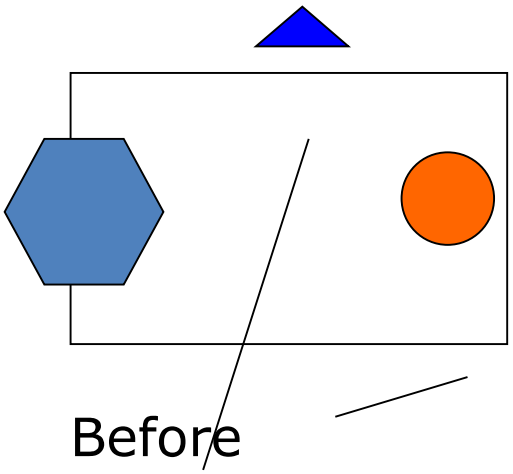
Illuminate pixel (round(x), y)

…

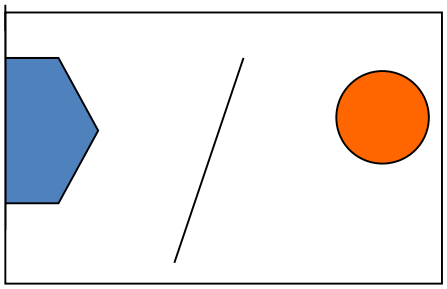
Until y == y1

Clipping

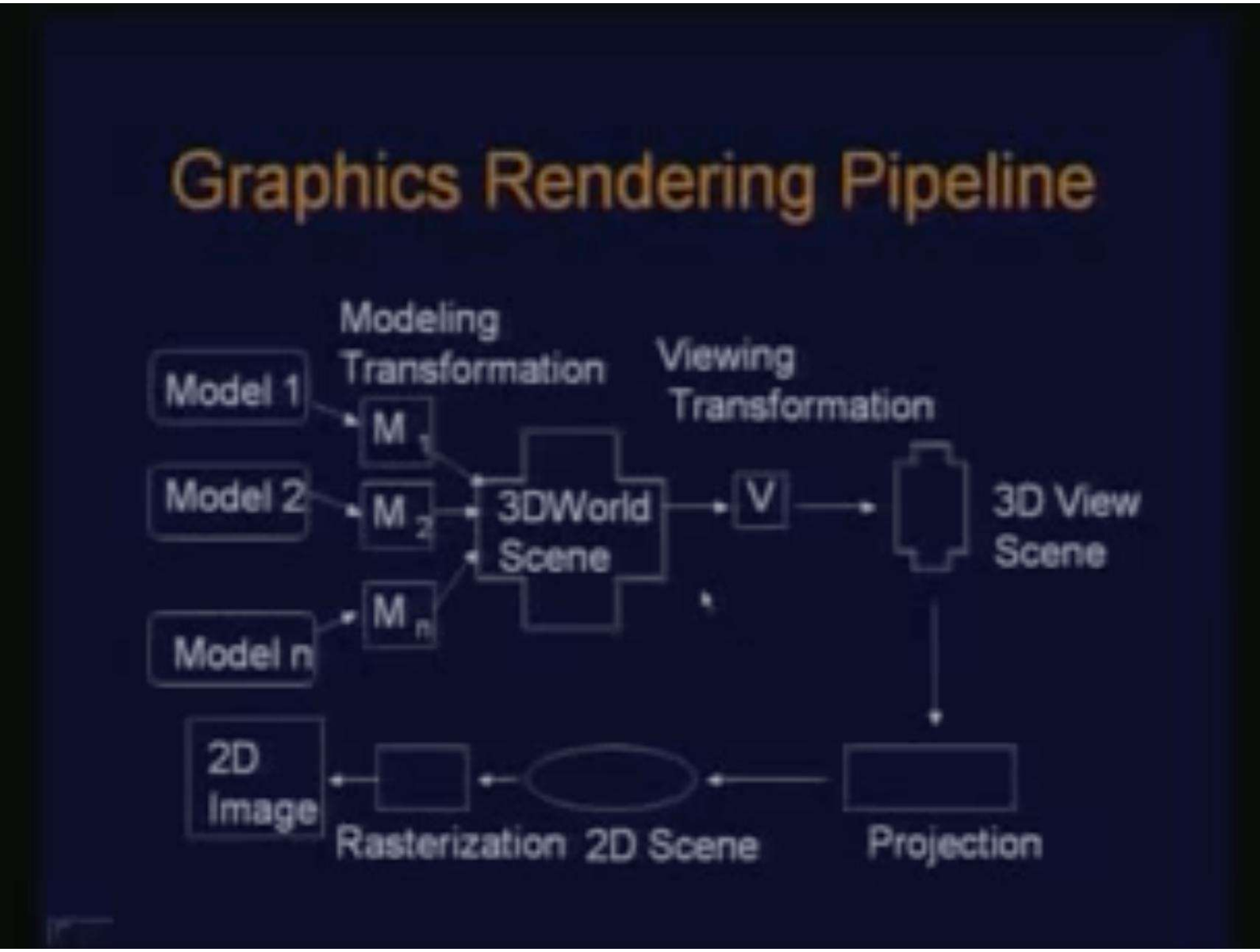
The graphics which exist outside the defined window area should be eliminated before the display on the viewport. This process of elimination of graphics which exist outside the window area is called clipping.



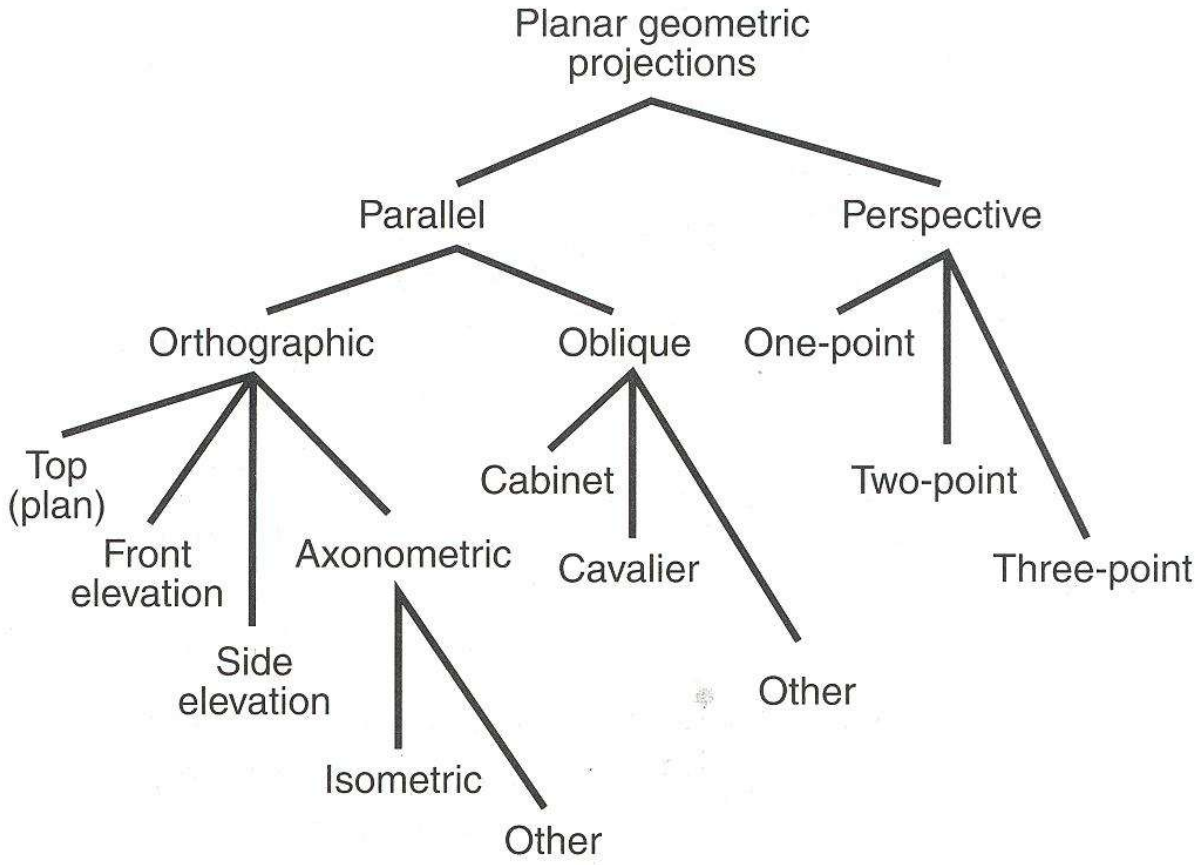
Before



After

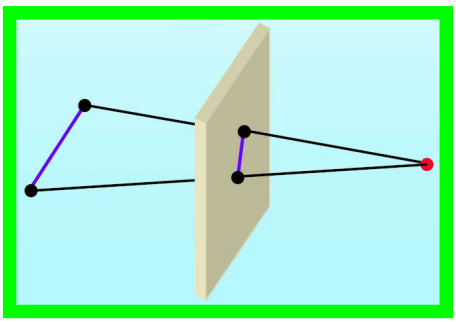

Graphics Rendering Pipeline
Modeling Transformation
Viewing Transformation
Model 1
Model 2
Model n
M 1
M 2
M n
3DWorld Scene
V
3D View Scene
2D Image
Rasterization
2D Scene
Projection

Taxonomy of Projections



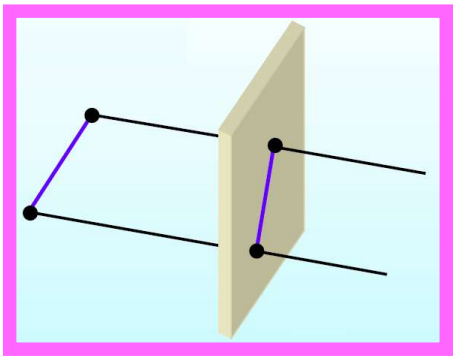
Perspective and Parallel Projection

perspective



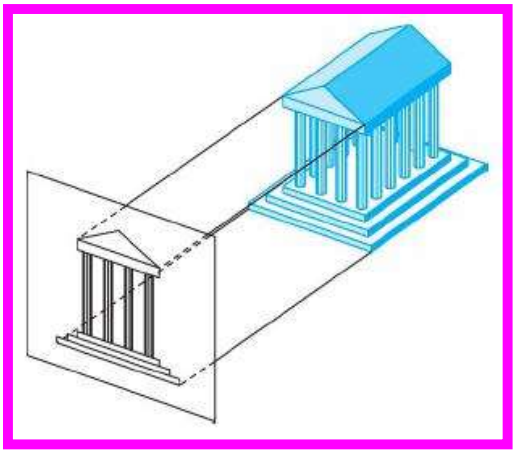
Perspective Projection: Object positions are transformed to the view plane along lines that converge to the **projection reference (center) point**.

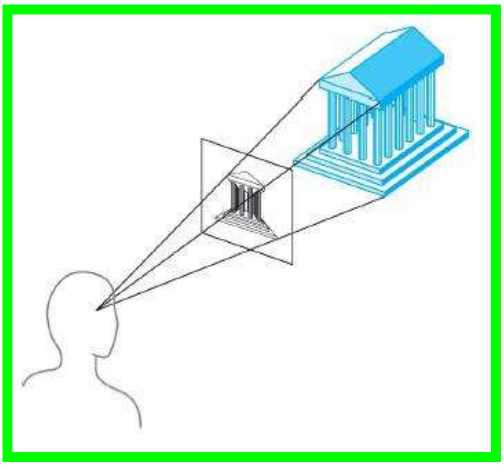
parallel

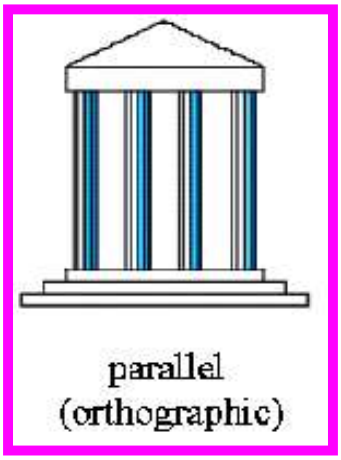


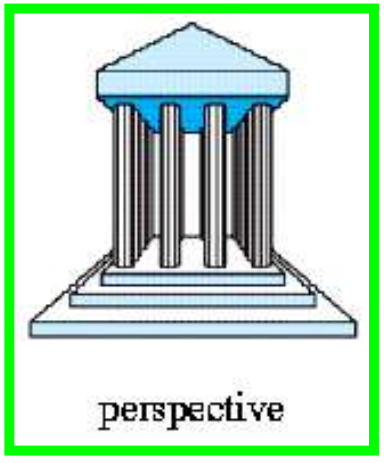
Parallel Projection: Coordinate position are transformed to the view plane along **parallel lines**.

Parallel and Perspective Projection



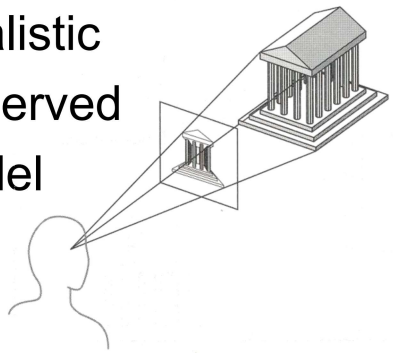


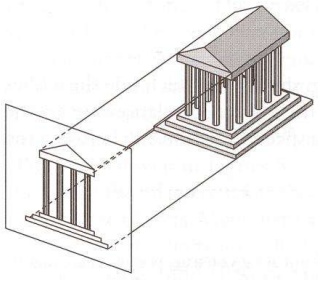




Perspective vs. Parallel

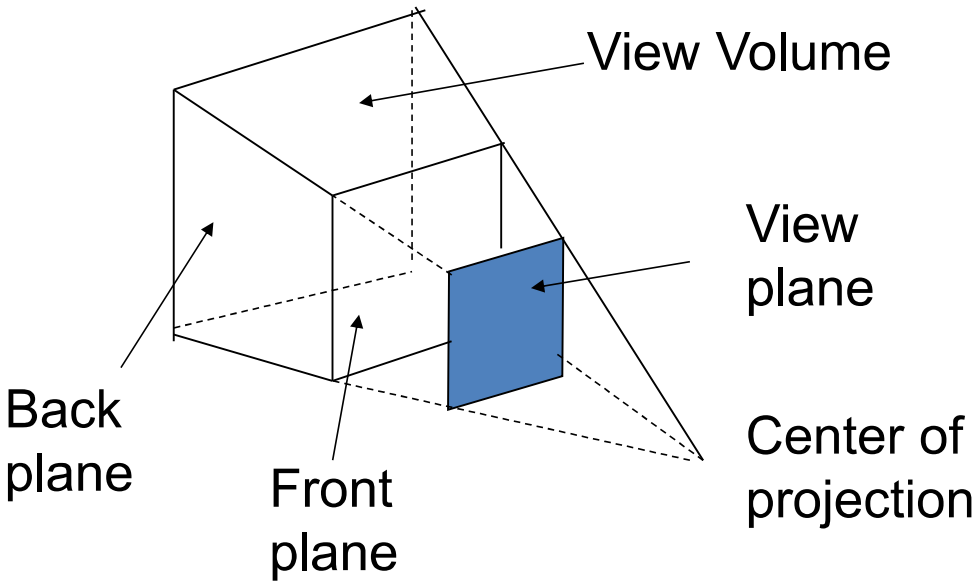
- Perspective projection
  - + Size varies inversely with distance - looks realistic
  - – Distance and angles are not (in general) preserved
  - – Parallel lines do not (in general) remain parallel
- Parallel projection
  - + Good for exact measurements
  - + Parallel lines remain parallel
  - – Angles are not (in general) preserved
  - – Less realistic looking

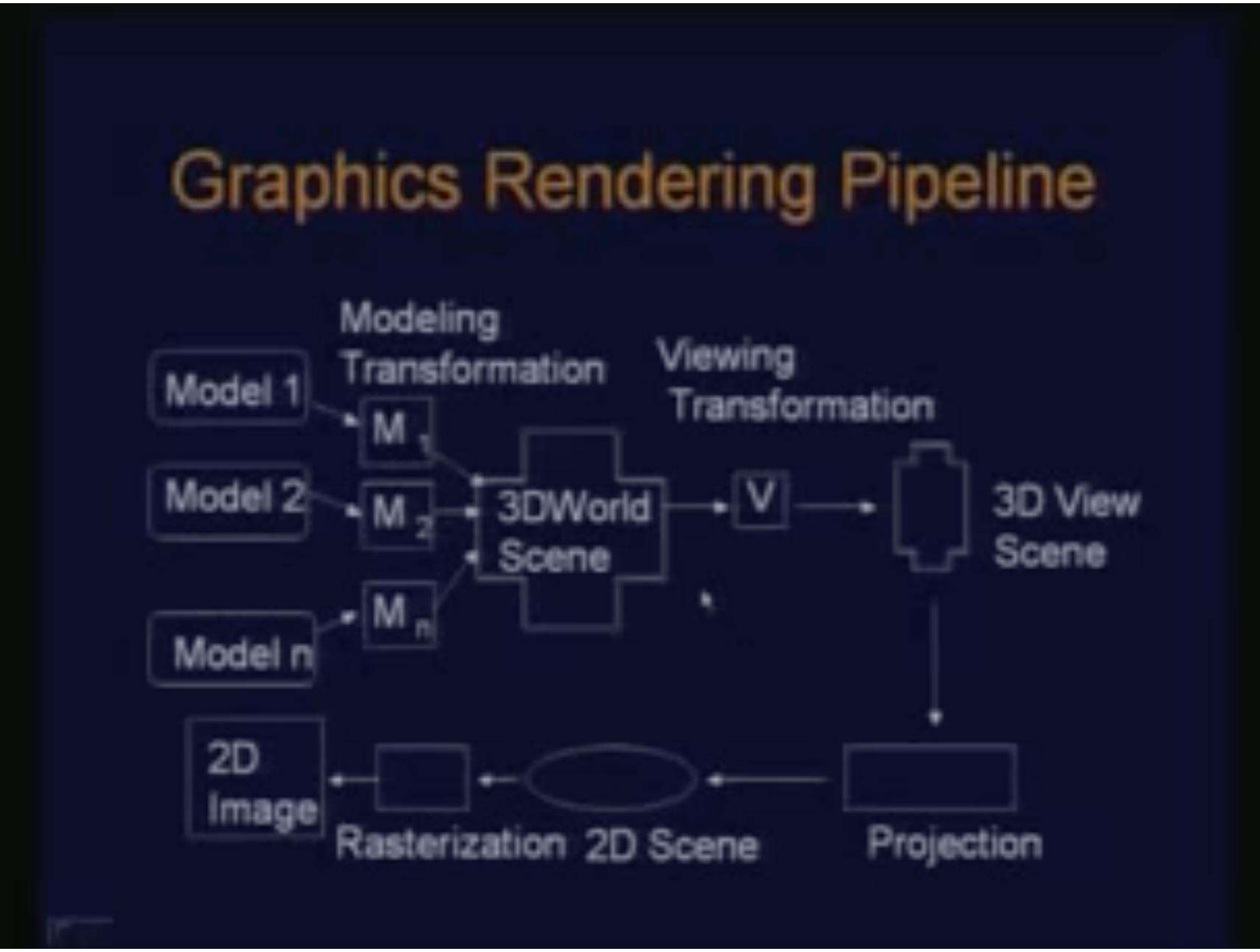




3D View Volumes

Perspective Projection




Graphics Rendering Pipeline
Modeling Transformation
Viewing Transformation
Model 1
Model 2
Model n
M 1
M 2
M n
3DWorld Scene
V
3D View Scene
2D Image
Rasterization
2D Scene
Projection

Graphics Rendering Pipeline

Stage	Process	Output
1. Models	Collection of 3D objects (Model 1, Model 2, …, Model n)	Raw 3D models
2. Modeling Transformation (M_1, M_2 … M_n)	Places each model into a **common world coordinate system** (scaling, rotation, translation).	Objects in a unified 3D world
3. 3D World Scene	Combination of all transformed models into a single scene.	Complete 3D world scene
4. Viewing Transformation (V)	Converts world coordinates into **camera (view) coordinates**.	3D scene from the camera's viewpoint
5. 3D View Scene	Scene as observed by the virtual camera.	Camera-centered 3D scene
6. Projection	Converts 3D view into 2D (Orthographic or Perspective).	Flattened 2D scene
7. 2D Scene	2D representation of the scene after projection.	Geometry in 2D space
8. Rasterization	Converts 2D primitives into pixels (fills with color, shading, textures).	Pixel-based representation
9. Final 2D Image	Display-ready image shown on the screen.	Rendered 2D image

DDA (Digital Differential Analyzer) Algorithm

- ✓ The line (vector) generation algorithms which determine the pixels that should be turned ON are called as digital differential analyzer (DDA).
- ✓ It is a simple and efficient algorithm used to draw lines on a computer screen.
- ✓ It is one of the technique for obtaining a rasterized straight line.
- ✓ This algorithm can be used to draw the line in all the quadrants.

Understanding The Algorithm

- ✓ DDA Algorithm is the simplest line drawing algorithm.
- ✓ Given the starting and ending coordinates of a line, DDA Algorithm attempts to generate the points between the starting and ending coordinates.

Procedure

- Given-
  - Starting coordinates = (X_0, Y_0)
  - Ending coordinates = (X_n, Y_n)

Procedure (Cont.)

- **Step-01:**

- Calculate ΔX, ΔY and M from the given input.
- These parameters are calculated as -

. $\Delta X = X_n - X_0$

. $\Delta Y = Y_n - Y_0$

. $M = \Delta Y / \Delta X$

Procedure (Cont.)

- **Step-02:**

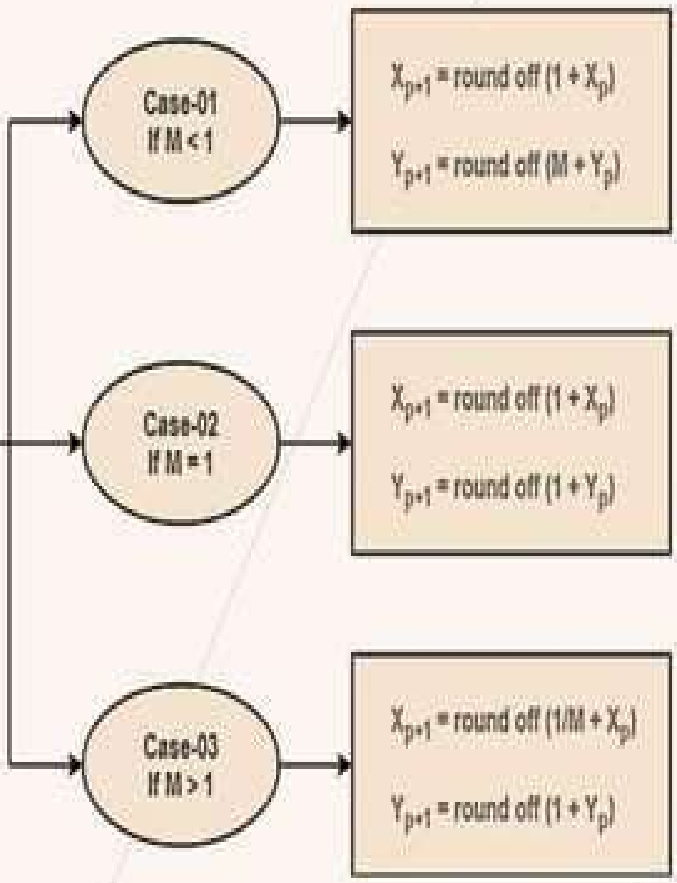
- Find the number of steps or points in between the starting and ending coordinates.

```
if (absolute (ΔX) > absolute (ΔY))
        Steps = absolute (ΔX);
else
        Steps = absolute (ΔY);
```

Procedure (Cont.)

- **Step-03:**

- Suppose the current point is (X_p, Y_p) and the next point is (X_{p+1}, Y_{p+1}).
- Find the next point by following the below three cases-



Procedure (Cont.)

- **Step-04:**

- Keep repeating **Step-03** until the end point is reached or the number of generated new points (including the starting and ending points) equals to the steps count.

Practice Problems Based On DDA Algorithm

- **Problem-01:**
- Calculate the points between the starting point (5, 6) and ending point (8, 12).

- **Solution-**
- Given-

. Starting coordinates = $(X_0, Y_0) = (5, 6)$

. Ending coordinates = $(X_n, Y_n) = (8, 12)$

Practice Problems Based On DDA Algorithm (cont.)

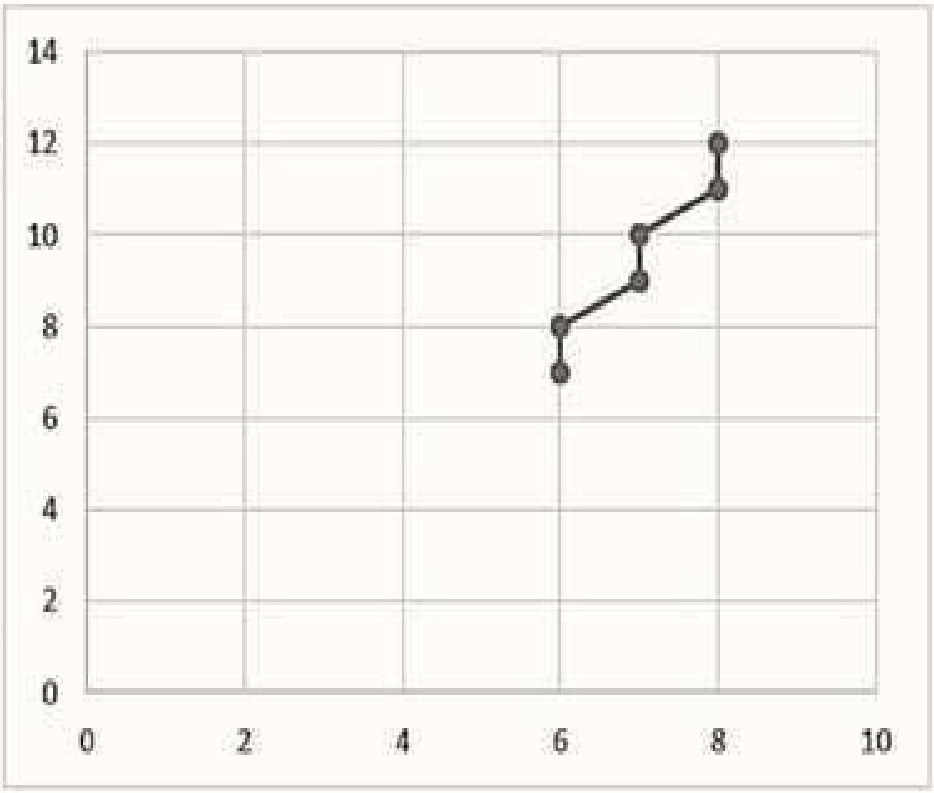
- **Step-01:**
- Calculate ΔX, ΔY and M from the given input.
  - $\Delta X = X_n - X_0 = 8 - 5 = 3$
  - $\Delta Y = Y_n - Y_0 = 12 - 6 = 6$
  - $M = \Delta Y / \Delta X = 6 / 3 = 2$
- **Step-02:**
  - Calculate the number of steps.
  - As $|\Delta X| < |\Delta Y| = 3 < 6$, so number of steps = $\Delta Y = 6$

Practice Problems Based On DDA Algorithm (cont.)

- Step-03:
- As M > 1, so case-03 is satisfied.
- Now, Step-03 is executed until Step-04 is satisfied.

X_p	Y_p	X_{p+1}	Y_{p+1}	Round off (X_{p+1}, Y_{p+1})
5	6	5.5	7	(6, 7)
		6	8	(6, 8)
		6.5	9	(7, 9)
		7	10	(7, 10)
		7.5	11	(8, 11)
		8	12	(8, 12)

Practice Problems Based On DDA Algorithm (cont.)



Advantages

- ✓The DDA algorithm is straight-forward to implement.
- ✓It only requires basic arithmetic operations like addition and division.
- ✓It can be used to draw lines in both positive and negative slopes.
- ✓DDA draws the line faster than drawing the line by directly using the line equation.

Limitations

- There is an extra overhead of using round off( ) function.
- Using round off( ) function increases time complexity of the algorithm.
- Resulted lines are not smooth because of round off( ) function.
- The points generated by this algorithm are not accurate.

Conclusion

- ✓ The DDA line drawing algorithm is a simple and widely used method to draw lines on raster displays.
- ✓ While it has its limitations, it remains a valuable tool for many applications.

Question 1

- Using the DDA (Digital Differential Analyzer) algorithm, draw a line between the points (5, 4) and (12, 7).

  Instructions:

  - Create a table showing each step of the DDA algorithm:
  - Include columns for: Step No, X, Y, Rounded X, Rounded Y
  - Round the X and Y values to the nearest integer to determine the pixels to be plotted.
  - Use graph paper to plot and draw the line by marking the rounded coordinates from your table.
  - Clearly label the starting and ending points.

Question

- Using the DDA (Digital Differential Analyzer) algorithm, draw a line between the points (5, 7) and (10, 15).

  Instructions:

  - Create a table showing each step of the DDA algorithm:
  - Include columns for: Step No, X, Y, Rounded X, Rounded Y
  - Round the X and Y values to the nearest integer to determine the pixels to be plotted.
  - Use graph paper to plot and draw the line by marking the rounded coordinates from your table.
  - Clearly label the starting and ending points.